# Chef Design Document

Oct 22, 2026 · @Everett Lopez

*Chef — your home cooking helper*

CS 4000 Capstone Project · Fall 2026 · Version 2.0 (supersedes v1.0 of October 5, 2026)

Team: Zach Smilor · Jason Khong · Everett Lopez · Max Donaldson

**What changed since v1.0:** adds requirements and prototype scope, a system architecture, a full data model with an ER diagram and table definitions, a data pipelines section for the data science component, and a recommendation approach. The front end is now specified as React with Tailwind CSS, code samples are corrected, and open decisions are listed at the end.

## Abstract

Chef is a recipe and pantry app that suggests meals from the ingredients a household already has. People buy groceries without a plan for combining them, and items with different expiration dates are easily forgotten and wasted. This hits busy people, college students and roommates hardest, since no one knows exactly what food is on hand.

Chef keeps a virtual pantry, shared across a household, and ranks recipes by how many of their ingredients are already in it. Ingredients nearing their expected expiration dates get priority, and dietary restrictions are respected. The distinctive idea is inventory-first discovery: instead of finding a recipe and then checking the pantry, the pantry finds the recipe. Chef aims to make meal planning easier while reducing food waste.

## Executive summary

Chef combines a virtual pantry with recipe recommendations, free of charge, so home cooks know what they have, what to make and what to use first.

It works in a few steps:

- **Fill the pantry.** Cooks scan a grocery receipt or add items by hand. Chef estimates an expiration date for each item.
- **Get suggestions.** Chef ranks recipes by how much of each recipe the pantry already covers, favoring ingredients that expire soon.

- **Log meals.** When a cook logs a recipe, Chef deducts the ingredients used from the pantry, oldest first.

Competitors split this job in two. NYT Cooking and Paprika charge for the app or a subscription. Tasty offers strong visuals and search but no pantry. Chef fills that gap with search, pantry tracking and a polished experience at no cost.

The project suits our team because it is full-stack. A polished front end will push our UI skills, and the back end needs real data work: ingredient normalization, a recipe ingestion pipeline and a recommendation model. Success means building something everyday home cooks actually want to use.

# Background

## Problem and solution space

Home cooks lose track of what is in their pantry. Food expires unused, duplicates get bought, and finding a recipe that uses up leftovers is tedious. Chef addresses all three by pairing pantry tracking with pantry-aware recipe suggestions. It is built for cooking, kitchen management and grocery shopping.

Recipe apps are common; pantry tracking is rarer, and the two are seldom combined.

| App | Recipe search | Pantry tracking | Cost |
|---|---|---|---|
| NYT Cooking | Yes | No | Subscription |
| Paprika | Yes (recipe manager) | Basic | Paid app |
| Tasty | Yes, strong visuals | No | Free |
| Epicurious | Yes | No | Subscription for full access |
| America's Test Kitchen | Yes | No | Subscription |
| **Chef** | **Yes** | **Yes, with expiry and shared households** | **Free** |

*Competitor details are approximate and may change over time.*

### Users and stakeholders

Primary users are people who want to spend less time planning meals and managing groceries:

- College students and others with limited free time
- People who often struggle to decide what to cook
- People who want to reduce food waste
- Households that shop for groceries regularly
- Roommates who need a shared view of the food on hand

Secondary stakeholders are the recipe and ingredient data providers Chef depends on, such as Spoonacular and the RecipeNLG dataset, and the USDA FoodKeeper data used for shelf life.

## Requirements and prototype scope

The prototype proves the core loop: stock a shared pantry, get ranked recipe suggestions, and log a meal that updates the pantry. Receipt scanning runs as a parallel, time-boxed spike because it is the riskiest feature.

### Functional requirements

| ID | Requirement | Prototype tier |
|---|---|---|
| F1 | Users sign up, log in and log out | MVP |
| F2 | Each user belongs to a household; others join with an invite code | MVP |
| F3 | Users add, edit and remove pantry items, with ingredient autocomplete | MVP |
| F4 | Each pantry item gets an estimated expiration date the user can override | MVP |
| F5 | Users browse and search recipes from the imported dataset | MVP |
| F6 | "What can I make" ranks recipes by pantry coverage and expiry urgency | MVP |

| ID | Requirement | Prototype tier |
|---|---|---|
| F7 | Users set dietary preferences that filter recommendations | MVP |
| F8 | Logging a cooked recipe deducts ingredients from the pantry, and can be undone | MVP |
| F9 | Users upload a receipt photo, review the parsed items and add them to the pantry | Spike, then MVP if reliable |
| F10 | Users save favorite recipes | Stretch |
| F11 | Users rate cooked recipes | Stretch |
| F12 | Waste report: items discarded vs. used over time | Stretch |
| F13 | Users create their own recipes | Stretch |
| F14 | Mobile app | Future |

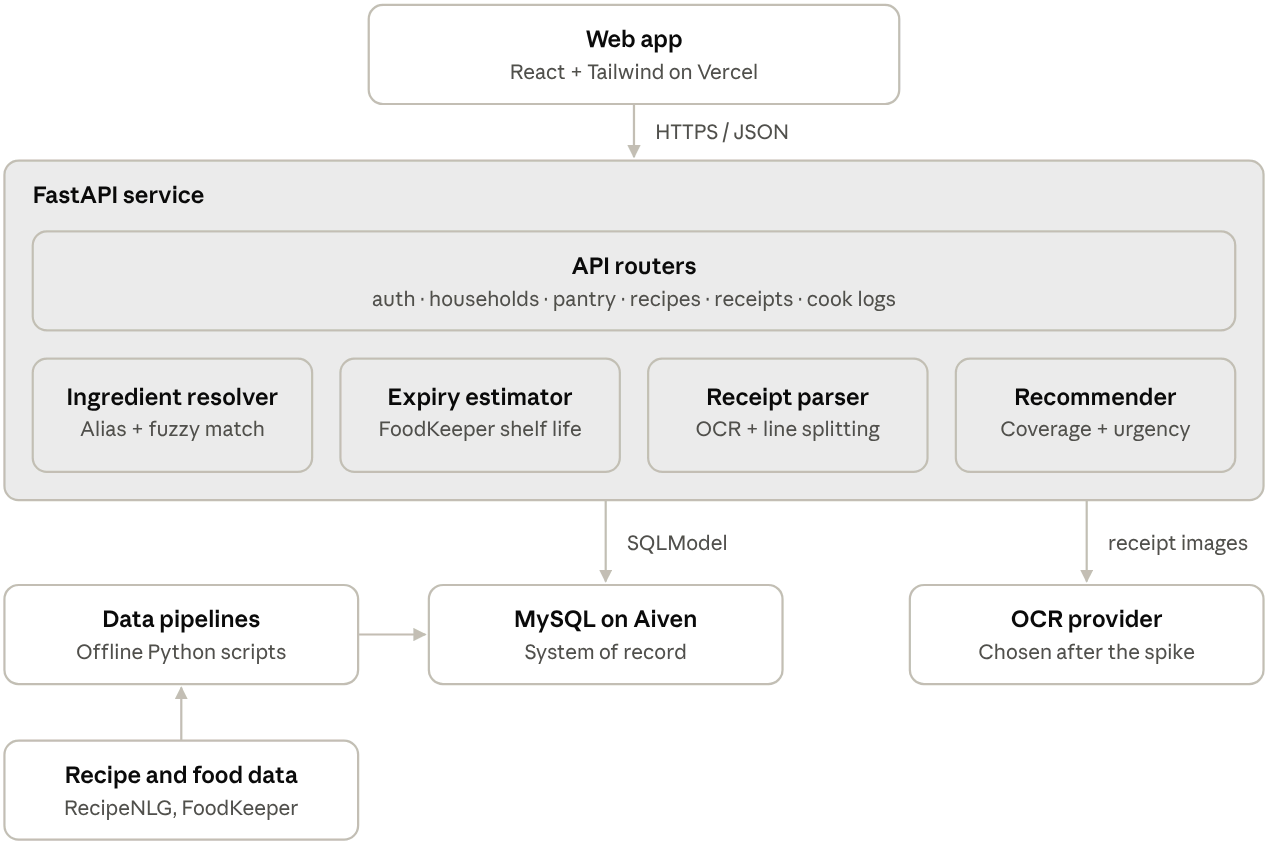
### Non-functional requirements

- **Usability:** adding a pantry item takes under 10 seconds with autocomplete.
- **Performance:** "What can I make" responds in under 2 seconds on the prototype dataset.
- **Security:** passwords are hashed (bcrypt or argon2); API routes require a valid session or token; a user can only read their own household's data.
- **Data quality:** pipeline metrics (see Data pipelines) are tracked and reported each milestone.
- **Responsiveness:** the web app is usable on phone-sized screens.

## System architecture

A React web app talks to a single FastAPI service, which owns all business logic and is the only component that writes to MySQL at runtime. Offline pipelines load and clean recipe data separately, so heavy processing never slows the live app.

**One FastAPI service sits between the web app and MySQL**




System architecture · web app, API service, database, pipelines

Routers handle HTTP concerns only; the four services hold the logic and are shared. The ingredient resolver is used both by the API (receipts, pantry autocomplete) and by the offline pipelines, so matching behaves the same everywhere.

## Technology stack

Chef uses a Python back end (FastAPI, SQLModel), a MySQL database hosted on Aiven, and a React front end styled with Tailwind CSS, deployed on Vercel.

| Layer | Choice | Role |
|---|---|---|
| Front end | React + Tailwind CSS | Web UI and styling, deployed on Vercel |
| API | FastAPI | HTTP endpoints, organized into routers |

| Layer | Choice | Role |
|---|---|---|
| Validation | Pydantic | Request and response schemas |
| ORM | SQLModel | Python classes mapped to MySQL tables |
| Migrations | Alembic | Versioned schema changes |
| Database | MySQL on Aiven | Managed, cloud-hosted relational store |
| API hosting | To decide: Render, Railway, Fly.io or Vercel Python functions | Runs the FastAPI service |
| Data pipelines | Python, pandas, rapidfuzz, Jupyter | Recipe ingestion, normalization, evaluation |
| Receipt OCR | To decide after the spike: Tesseract, Google Cloud Vision or an LLM vision API | Receipt image to text |
| Source control | GitLab | Code, issues, CI |

## FastAPI

FastAPI defines the API endpoints that connect the front end to the services and the database.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Welcome to Chef"}
```

## Routers

Routers split endpoints into modules (`auth`, `households`, `pantry`, `recipes`, `receipts`, `cook_logs`), which keeps the app maintainable.

```
from fastapi import APIRouter, Depends
from sqlmodel import Session, select

from app.db import get_session
from app.models import Recipe

router = APIRouter(prefix="/recipes", tags=["recipes"])

@router.get("/")
def list_recipes(session: Session = Depends(get_session)):
    return session.exec(select(Recipe).limit(50)).all()
```

### Pydantic

Pydantic validates incoming requests before they reach business logic.

```
from pydantic import BaseModel

class RecipeCreate(BaseModel):
    title: str
    description: str
    prep_time_min: int
```

### SQLModel

SQLModel defines tables as Python classes, combining SQLAlchemy and Pydantic with little boilerplate.

```
from sqlmodel import SQLModel, Field

class Recipe(SQLModel, table=True):
    __tablename__ = "recipes"

    id: int | None = Field(default=None, primary_key=True)
    title: str = Field(max_length=200, index=True)
    description: str | None = None
```

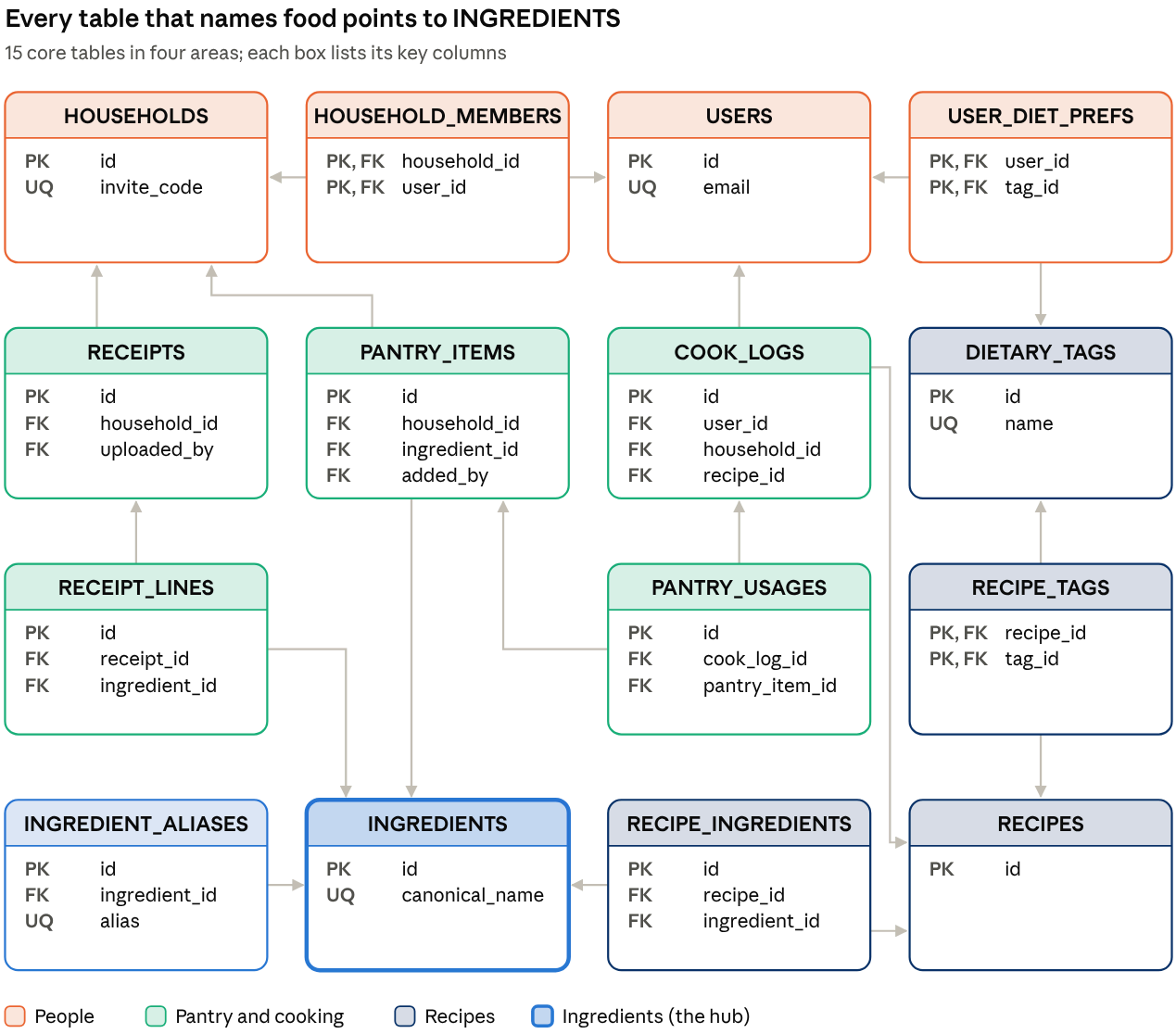
### MySQL on Aiven

MySQL is the system of record. Aiven hosts it as a managed service, so the team shares one cloud database without maintaining infrastructure.

```
CREATE TABLE recipes (
  id INT PRIMARY KEY AUTO_INCREMENT,
  title VARCHAR(200) NOT NULL,
  description TEXT
);
```

## Data model

The schema centers on one canonical `INGREDIENTS` table: pantry items, recipe ingredients and receipt lines all point to it, so "what can I make" becomes a join instead of a text-matching problem.




Entity relationship diagram · 15 core tables, foreign keys as arrows

Colors group tables by area. Households own pantries and receipts, and cook logs connect users, recipes and the pantry batches a meal drew from.

## Table definitions

Tables are grouped by area, in the same order and colors as the diagram. A composite primary key is written in parentheses; each foreign key names the table it references.

| Area | Table | Primary key | Foreign keys | Other attributes |
|---|---|---|---|---|
| People | USERS | id | — | email (unique), password_hash, display_name, created_at |
| People | HOUSEHOLDS | id | — | name, invite_code (unique) |
| People | HOUSEHOLD_MEMBERS | (household_id, user_id) | household_id → HOUSEHOLDS; user_id → USERS | role |
| People | USER_DIET_PREFS | (user_id, tag_id) | user_id → USERS; tag_id → DIETARY_TAGS | mode |
| Pantry | PANTRY_ITEMS | id | household_id → HOUSEHOLDS; ingredient_id → INGREDIENTS; added_by → USERS | quantity, unit, storage, purchased_on, expires_on, expiry_estimated, status |
| Pantry | RECEIPTS | id | household_id → HOUSEHOLDS; uploaded_by → USERS | image_url, raw_ocr, status |
| Pantry | RECEIPT_LINES | id | receipt_id → RECEIPTS; ingredient_id → INGREDIENTS (nullable until matched) | raw_text, quantity, match_confidence, match_method, confirmed |
| Pantry | COOK_LOGS | id | user_id → USERS; household_id → HOUSEHOLDS; recipe_id → RECIPES | servings_made, rating, cooked_at |
| Pantry | PANTRY_USAGES | id | cook_log_id → COOK_LOGS; pantry_item_id → PANTRY_ITEMS | quantity_used |

| Area | Table | Primary key | Foreign keys | Other attributes |
|---|---|---|---|---|
| Ingredients | INGREDIENTS | id | — | canonical_name (unique), category, default_unit, shelf_days_pantry, shelf_days_fridge, shelf_days_freezer |
| Ingredients | INGREDIENT_ALIASES | id | ingredient_id → INGREDIENTS | alias (unique), source |
| Recipes | RECIPES | id | — | title, instructions, prep_min, cook_min, servings, source, external_id |
| Recipes | RECIPE_INGREDIENTS | id | recipe_id → RECIPES; ingredient_id → INGREDIENTS | quantity, unit, raw_text, optional, match_confidence, match_method |
| Recipes | DIETARY_TAGS | id | — | name (unique), kind |
| Recipes | RECIPE_TAGS | (recipe_id, tag_id) | recipe_id → RECIPES; tag_id → DIETARY_TAGS | — |
| Recipes | SAVED_RECIPES (stretch) | (user_id, recipe_id) | user_id → USERS; recipe_id → RECIPES | saved_at |

The same schema as Mermaid source, for diagram tools, is in the appendix.

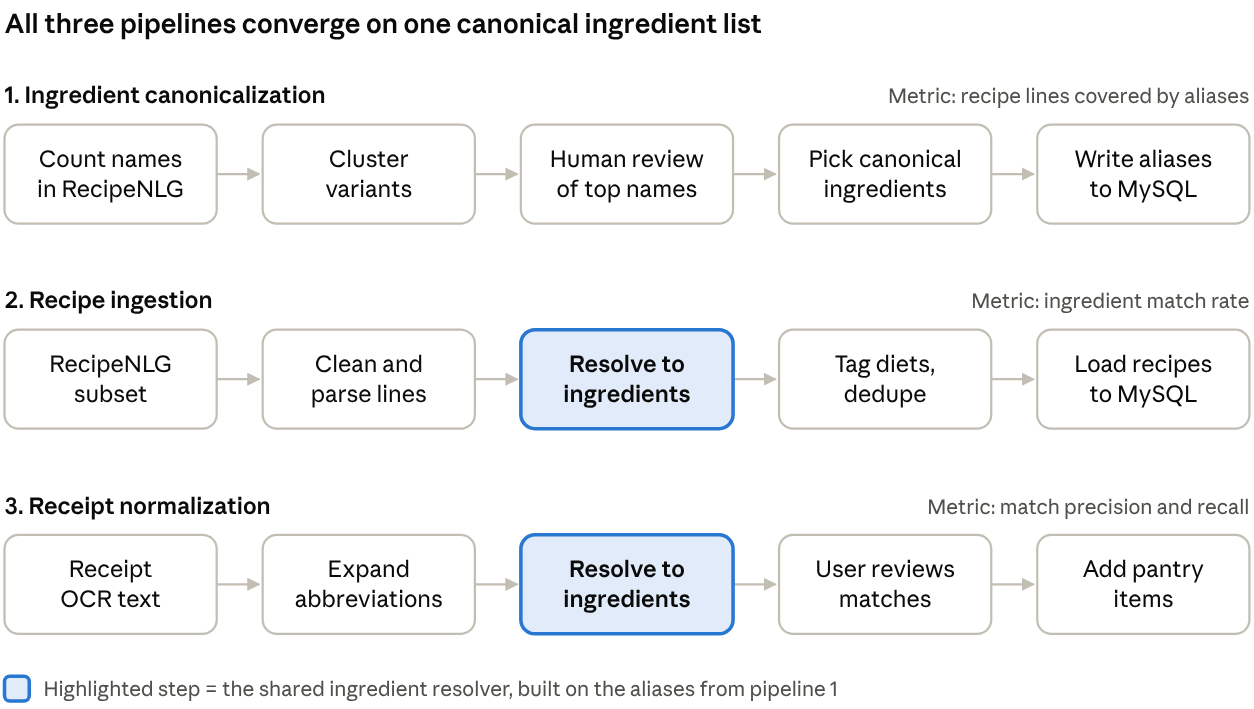
## Design decisions

- **Ingredients are the hub.** Pantry items, recipe ingredients and receipt lines all reference one canonical ingredient, so matching happens once.
- **Pantries belong to households.** A solo user gets a household of one at signup; roommates join with an invite code.
- **Pantry items are batches, not totals.** Each purchase has its own expiration date, and `status` (active, used, discarded) keeps history for a future waste report.
- **Receipts are staged.** Only receipt lines a user confirms become pantry items.
- **Cook logs are reversible.** `PANTRY_USAGES` records which batches a meal drew from, oldest-expiring first, so a log can be undone.

- **Matches are auditable.** Recipe and receipt lines keep their raw text plus `match_confidence` and `match_method`.
- **Allergens are a stretch goal.** An `INGREDIENT_ALLERGENS` table would let allergen filtering work per ingredient.

## Data pipelines and normalization

Chef's data science component is ingredient normalization: turning messy text from recipes and receipts into canonical ingredients, with every pipeline measured against a labeled evaluation set. The same text arrives in many forms ("GV 2% MLK GAL" on a receipt, "2 cups whole milk" in a recipe), and recommendations only work once they resolve to the same `INGREDIENTS` row.




Data pipelines · three flows sharing one ingredient resolver

Pipeline 1 runs first because it builds the vocabulary the other two match against. Pipelines 2 and 3 share one resolver: exact alias lookup, then fuzzy matching, each match stored with `match_confidence` and `match_method`.

## Pipeline details

| Pipeline | Input | Output | Methods | Evaluation |
|---|---|---|---|---|
| 1. Ingredient canonicalization | RecipeNLG ingredient-name field | `INGREDIENTS`, `INGREDIENT_ALIASES` | Frequency analysis; fuzzy matching (rapidfuzz) or text embeddings to cluster variants; human review of top clusters | Share of recipe ingredient lines covered by an alias |
| 2. Recipe ingestion | RecipeNLG subset (20,000–50,000 recipes) | `RECIPES`, `RECIPE_INGREDIENTS`, `RECIPE_TAGS` | Text cleaning; quantity, unit and name parsing; resolution; near-duplicate detection; rule-based diet tagging | Match rate on all lines; accuracy on 200 hand-labeled lines |
| 3. Receipt normalization | OCR text from receipt images | `RECEIPT_LINES`, then `PANTRY_ITEMS` | Line splitting; abbreviation expansion; resolution; confidence threshold for review | Precision and recall on a labeled set of real receipts |

| Pipeline | Input | Output | Methods | Evaluation |
|---|---|---|---|---|
| 4. Expiration estimation | USDA FoodKeeper shelf-life data | Shelf-life days on `INGREDIENTS` | Map FoodKeeper foods to canonical ingredients | Stretch: compare estimates with user overrides and discarded items |

## Parsing example

| Raw line | Quantity | Unit | Ingredient | Method |
|---|---|---|---|---|
| 2 cups chopped onion | 2 | cup | onion | exact alias |
| 1 (14 oz) can diced tomatoes | 14 | oz | canned tomatoes | fuzzy |
| GV 2% MLK GAL (receipt) | 1 | gallon | milk | fuzzy, user confirmed |

## Feedback loop

When a user corrects or confirms a receipt match, the confirmed text is saved as a new alias. Matching improves with use, and the corrections become fresh labeled data for evaluating pipeline 3.

## Tools and practices

- **Analysis:** Jupyter notebooks for exploration and metric reports.
- **Repeatable runs:** Python scripts in the repo ( `pipelines/` ), re-runnable from scratch against a dev database.
- **Libraries:** pandas, rapidfuzz; sentence embeddings if fuzzy matching falls short.
- **Reporting:** match rates and precision/recall reported at each milestone.
- **Ownership:** led by Zach Smilor (data science major), with the back-end developer owning the shared resolver's API integration.

## Recommendation approach

The prototype ranks recipes with a transparent SQL-based score, not a trained model, so results are explainable and easy to tune. Machine learning can replace parts of it later if evaluation shows it helps.

For a household and a recipe, the score combines two terms:

```
\text{score} = w_c \cdot \frac{\text{required ingredients in pantry}}
{\text{required ingredients}} + w_u \cdot \frac{\text{pantry ingredients used
that expire within } d \text{ days}}{\text{required ingredients}}
```

- **Coverage** rewards recipes the household can make now. Optional ingredients and staples like salt and water are excluded from the count.
- **Urgency** rewards recipes that use items about to expire.
- **Weights** (w_c, w_u) and the window d start at 0.7, 0.3 and 3 days, then get tuned during evaluation.

Recipes that violate a user's dietary preferences are filtered out before scoring. Results can also be filtered by "missing at most N ingredients," which doubles as a shopping hint.

**Evaluation:** the team writes about 20 pantry scenarios (e.g., "chicken, rice, spinach expiring tomorrow") and judges whether the top 5 results are sensible. Precision@5 across scenarios is the tracked metric.

## External data sources

Recipes come primarily from a curated RecipeNLG subset; Spoonacular is optional enrichment, pending a review of its terms.

| Source | Provides | Constraints to verify |
|---|---|---|
| RecipeNLG | About 2.2M recipes with ingredient lines, directions and an extracted ingredient-name field | Non-commercial research and education use; we import a 20,000–50,000 recipe subset |
| Spoonacular API | Recipe search, nutrition, ingredient metadata | Free-tier request quotas; terms restrict long-term caching of its data |
| USDA FoodKeeper | Shelf life by food and storage location | Public data; confirm current format |

Each source's data quality, license, usage limits and cost will be confirmed before integration.

## Deployment

The prototype is a web app with no special hardware requirements.

- **Front end:** React + Tailwind CSS on Vercel, with preview deploys per merge request.
- **API:** FastAPI on a cloud host (to decide).
- **Database:** MySQL on Aiven; separate dev and prod databases.
- **Pipelines:** run on demand from the repo by a script, not on the web server.
- **Source control and CI:** GitLab, running tests and linting on each merge request.

A mobile app is a future goal; its approach will be evaluated once the web prototype is stable.

## Risks

| Risk | Impact | Mitigation |
|---|---|---|
| Receipt OCR is unreliable | Pantry entry stays manual | Time-boxed spike; mandatory review screen; manual entry with autocomplete as fallback |
| Ingredient matching is poor | Bad recommendations | Measured match rate; alias table grows from user corrections |

| Risk | Impact | Mitigation |
|---|---|---|
| Unit conversion is complex | Wrong deductions after cooking | Allow quantity-less pantry items ("have some"); convert only common units |
| Data licensing limits use | Must replace a data source | Use RecipeNLG as the primary source; keep Spoonacular optional |
| Free-tier hosting limits | Slow or sleeping API | Choose a host early and load-test the recommender |

## Open decisions

- [ ] Pick the API host (Render, Railway, Fly.io or Vercel functions)
- [ ] Choose the OCR approach after the receipt spike
- [ ] Decide quantity precision: exact amounts or "have / low / out"
- [ ] Confirm Spoonacular terms and whether to use it at all
- [ ] Set the RecipeNLG subset size and selection criteria
- [ ] Decide whether users can create their own recipes in v1

## Appendix: ER diagram source (Mermaid)

Paste this into any Mermaid renderer (GitLab markdown, mermaid.live) to regenerate the full ER diagram with every column.

```
erDiagram
  USERS ||--o{ HOUSEHOLD_MEMBERS : joins
  HOUSEHOLDS ||--o{ HOUSEHOLD_MEMBERS : has
  HOUSEHOLDS ||--o{ PANTRY_ITEMS : stocks
  INGREDIENTS ||--o{ PANTRY_ITEMS : "stocked as"
  INGREDIENTS ||--o{ INGREDIENT_ALIASES : "known as"
  INGREDIENTS ||--o{ RECIPE_INGREDIENTS : "used in"
  RECIPES ||--o{ RECIPE_INGREDIENTS : requires
  HOUSEHOLDS ||--o{ RECEIPTS : uploads
  RECEIPTS ||--o{ RECEIPT_LINES : contains
  INGREDIENTS |o--o{ RECEIPT_LINES : "matched to"
  USERS ||--o{ COOK_LOGS : cooks
  HOUSEHOLDS ||--o{ COOK_LOGS : "cooked in"
  RECIPES ||--o{ COOK_LOGS : "logged in"
```

```
  COOK_LOGS ||--o{ PANTRY_USAGES : deducts
  PANTRY_ITEMS ||--o{ PANTRY_USAGES : "drawn from"
  RECIPES ||--o{ RECIPE_TAGS : tagged
  DIETARY_TAGS ||--o{ RECIPE_TAGS : labels
  USERS ||--o{ USER_DIET_PREFS : sets
  DIETARY_TAGS ||--o{ USER_DIET_PREFS : "applies to"
  USERS ||--o{ SAVED_RECIPES : saves
  RECIPES ||--o{ SAVED_RECIPES : "saved as"
  USERS {
    int id PK
    string email UK
    string password_hash
    string display_name
    datetime created_at
  }
  HOUSEHOLDS {
    int id PK
    string name
    string invite_code UK
  }
  HOUSEHOLD_MEMBERS {
    int household_id PK,FK
    int user_id PK,FK
    string role
  }
  INGREDIENTS {
    int id PK
    string canonical_name UK
    string category
    string default_unit
    int shelf_days_pantry
    int shelf_days_fridge
    int shelf_days_freezer
  }
  INGREDIENT_ALIASES {
    int id PK
    int ingredient_id FK
    string alias UK
    string source
  }
  PANTRY_ITEMS {
    int id PK
    int household_id FK
```

```
    int ingredient_id FK
    decimal quantity
    string unit
    string storage
    date purchased_on
    date expires_on
    bool expiry_estimated
    string status
    int added_by FK
  }
  RECIPES {
    int id PK
    string title
    text instructions
    int prep_min
    int cook_min
    int servings
    string source
    string external_id
  }
  RECIPE_INGREDIENTS {
    int id PK
    int recipe_id FK
    int ingredient_id FK
    decimal quantity
    string unit
    string raw_text
    bool optional
    float match_confidence
    string match_method
  }
  RECEIPTS {
    int id PK
    int household_id FK
    int uploaded_by FK
    string image_url
    text raw_ocr
    string status
  }
  RECEIPT_LINES {
    int id PK
    int receipt_id FK
    int ingredient_id FK
```

```
  string raw_text
  decimal quantity
  float match_confidence
  string match_method
  bool confirmed
}
COOK_LOGS {
  int id PK
  int user_id FK
  int household_id FK
  int recipe_id FK
  int servings_made
  int rating
  datetime cooked_at
}
PANTRY_USAGES {
  int id PK
  int cook_log_id FK
  int pantry_item_id FK
  decimal quantity_used
}
DIETARY_TAGS {
  int id PK
  string name UK
  string kind
}
RECIPE_TAGS {
  int recipe_id PK,FK
  int tag_id PK,FK
}
USER_DIET_PREFS {
  int user_id PK,FK
  int tag_id PK,FK
  string mode
}
SAVED_RECIPES {
  int user_id PK,FK
  int recipe_id PK,FK
  datetime saved_at
}
```